

1. Provisioning

Script de provisioning d'utilisateurs avec validation.

```
Administrator: Windows PowerShell
PS C:\Users\Administrateur>
$Users = Import-Csv "C:\NouveauxUtilisateurs.csv"
foreach ($User in $Users) {
    try {
        if (-not (Get-ADUser -Filter "SamAccountName -eq '$($User.SamAccountName)')) {
            New-ADUser -Name $User.Name -SamAccountName $User.SamAccountName -UserPrincipalName
            $User.UPN -Enabled $true -Path $User.OU
            Write-Output "Utilisateur $($User.Name) créé avec succès"
        } else {
            Write-Warning "L'utilisateur $($User.SamAccountName) existe déjà"
        }
    } catch {
        Write-Error "Erreur lors de la création de $($User.Name): $($_.Exception.Message)"
    }
}
PS C:\Users\Administrateur> _
```

2. Groupes Dynamiques

Automatiser la gestion des groupes dynamiques.

```
Administrator: Windows PowerShell
PS C:\Users\Administrateur>
# Script pour mettre à jour les groupes basés sur les
attributs AD

$Departments = @("IT", "Finance", "RH")

foreach ($Dept in $Departments) {

    $GroupName = "Groupe_$Dept"

    $Members = Get-ADUser -Filter "Department -eq '$Dept'"
-Properties Department

    Set-ADGroup -Identity $GroupName -GroupCategory Security
-GroupScope DomainLocal

    Add-ADGroupMember -Identity $GroupName -Members
$Members
}
PS C:\Users\Administrateur> 
```

3. Inventaire AD

Script de rapport
d'inventaire AD
complet.

```
Administrator: Windows PowerShell
PS C:\Users\Administrateur>
$Report = @()
$Report += "=== INVENTAIRE AD - $(Get-Date) ==="
$Report += "Utilisateurs: $((Get-ADUser -Filter *).Count )"
$Report += "Ordinateurs: $((Get-ADComputer -Filter *).Count )"
$Report += "Groupes: $((Get-ADGroup -Filter *).Count )"
$Report += "OUs: $((Get-ADOrganizationalUnit -Filter *).Count )"
$Report | Out-File "C:\RapportInventaireAD.txt"
PS C:\Users\Administrateur> _
```

4. Licences

Automatiser la
gestion des
licences
logicielles via AD.

```
Administrator: Windows PowerShell
PS C:\Users\Administrateur>
# Script d'inventaire des logiciels installés

Get-ADComputer -Filter * | ForEach-Object {

    $Software = Invoke-Command -ComputerName $_.Name -ScriptBlock { Get-WmiObject -Class
Win32_Product | Select-Object Name, Version } -ErrorAction SilentlyContinue

    if ($Software) {

        $Software | Export-Csv "C:\InventaireLogiciels_$($_.Name).csv" -NoTypeInfoation

    }
}
PS C:\Users\Administrateur> █
```

5. Conformité

Script de conformité des politiques de sécurité.

Administrator: Windows PowerShell

```
PS C:\Users\Administrateur>
```

```
PS C:\Users\Administrateur> $ComplianceReport = @()  
$PasswordPolicy = Get-ADDefaultDomainPasswordPolicy
```

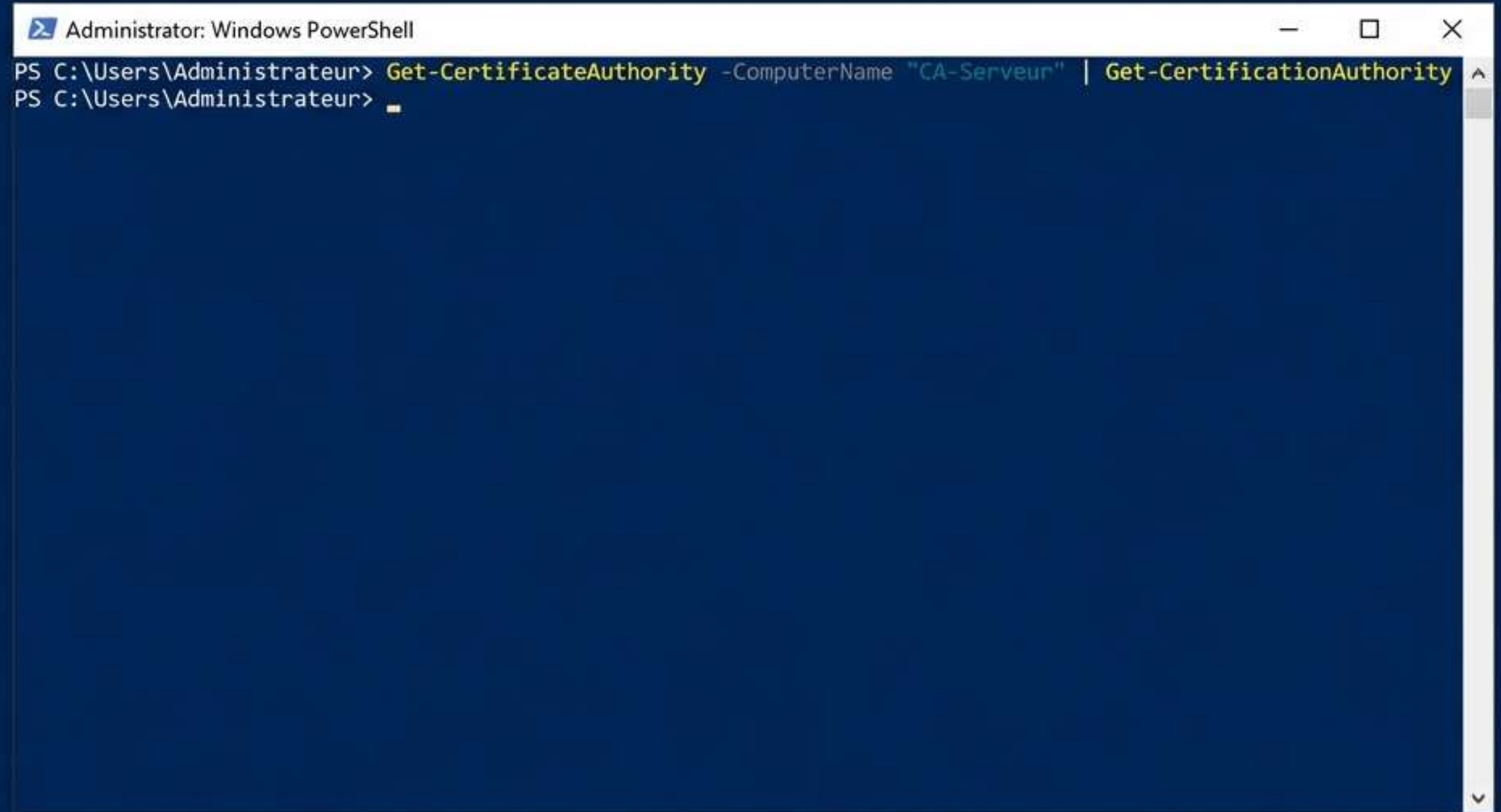
```
$ComplianceReport += "Durée minimale du mot de passe: $($PasswordPolicy.MinPasswordLength)"  
$ComplianceReport += "Durée minimale du mot de passe: $($PasswordPolicy.MinPasswordLength)"  
$ComplianceReport += "Complexité requise: $($PasswordPolicy.ComplexityEnabled)"
```

```
$ComplianceReport | Out-File "C:\RapportConformite.txt"
```

```
PS C:\Users\Administrateur> _
```

6. Certificats

Automatisez la
gestion des
certificats AD CS.



```
Administrator: Windows PowerShell
PS C:\Users\Administrateur> Get-CertificateAuthority -ComputerName "CA-Serveur" | Get-CertificationAuthority
PS C:\Users\Administrateur>
```

7. Migration

Script de migration
d'objets entre
domaines.

```
Administrator: Windows PowerShell
PS C:\Users\Administrateur>
# Utiliser ADMT (Active Directory Migration Tool) ou script personnalisé
$UsersToMigrate = Get-ADUser -Filter * -SearchBase "OU=AncienDomaine,DC=ancien,DC=lan"
foreach ($User in $UsersToMigrate) {
    # Logique de migration
}
PS C:\Users\Administrateur> .
```

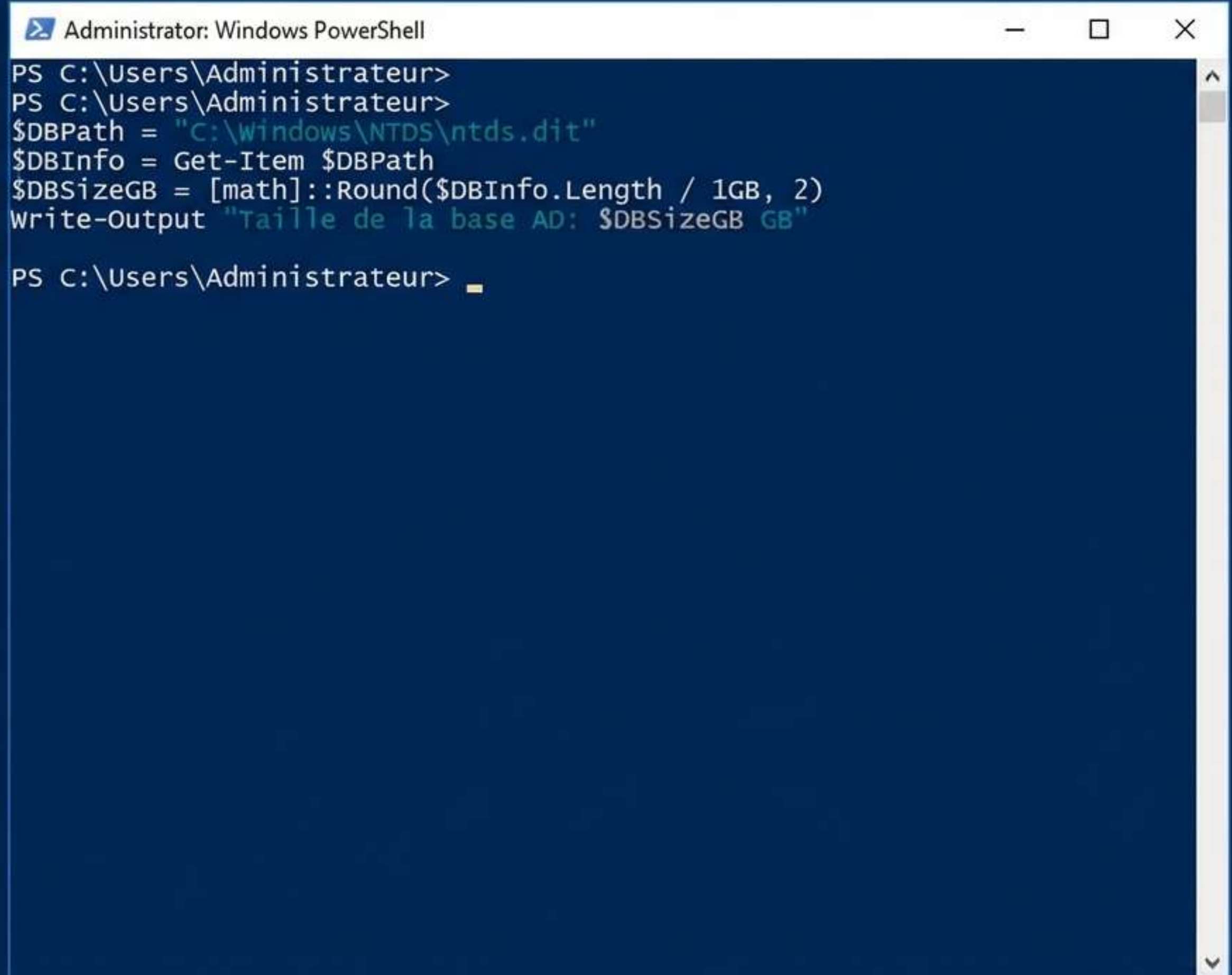
8. Sites/Subnets

Automatisez la gestion des sites et sous-réseaux.

```
Administrator: Windows PowerShell
PS C:\Users\Administrateur> $SitesConfig = Import-Csv "C:\ConfigurationSites.csv"
foreach ($Site in $SitesConfig) {
    New-ADReplicationSite -Name $Site.Nom
    New-ADReplicationSubnet -Name $Site.SousReseau -Site $Site.Nom
}
PS C:\Users\Administrateur> _
```

9. Monitoring

Script de monitoring de la capacité AD.



```
Administrator: Windows PowerShell
PS C:\Users\Administrateur>
PS C:\Users\Administrateur>
$DBPath = "C:\Windows\NTDS\ntds.dit"
$DBInfo = Get-Item $DBPath
$DBSizeGB = [math]::Round($DBInfo.Length / 1GB, 2)
Write-Output "Taille de la base AD: $DBSizeGB GB"
PS C:\Users\Administrateur> _
```

10. Sauvegardes

Automatisez les sauvegardes et restaurations.

Administrator: Windows PowerShell

```
PS C:\Users\Administrateur>
# Script de sauvegarde automatisé

$BackupPath = "C:\BackupAD\$(Get-Date -Format 'yyyyMMdd_HH:mm:ss')"

Backup-ADDS -Directory $BackupPath -Description "Sauvegarde_Automatique"
PS C:\Users\Administrateur> _
```